SALESIA ENTERPRISE · FASE 16

Manual técnico

Arquitectura, configuración, base de datos y operación

DOCUMENTO	manual-tecnico.md
VERSIÓN	1.0
FECHA	14 de octubre de 2026
PLATAFORMA	React 19 + TypeScript · FastAPI · PostgreSQL 17
DESPLIEGUE	Render (API) · Vercel (web) · Supabase (BD)
ESTADO	Sistema desplegado y verificado

Índice

1. Descripción general 3
2. Arquitectura 4
3. Estructura del repositorio 4
4. Configación y variables de entorno 5
5. Puesta en marcha en local 5
6. Base de datos 6
7. Motor estadístico e insights 6
8. Seguridad 7
9. Pruebas 7
10. Despliegue 7
11. Solución de problemas 7

- 1. Descripción general
- 2. Arquitectura
- 3. Estructura del repositorio
- 4. Configación y variables de entorno
- 5. Puesta en marcha en local
- 6. Base de datos
- 7. Motor estadístico e insights
- 8. Seguridad
- 9. Pruebas
- 10. Despliegue
- 11. Solución de problemas

Manual técnico — SalesIA Enterprise

> Documento operativo para el equipo de desarrollo y mantenimiento. Versión 1.0 · Fase 16.

1. Descripción general

SalesIA Enterprise es una aplicación web de gestión comercial con módulo de analítica estadística. Convierte la operación diaria (clientes, productos, ventas, pagos, inventario) en datos que el motor estadístico analiza para producir KPIs, gráficos, insights automáticos y reportes exportables.

Capa	Tecnología
Frontend	React 19 + TypeScript + Vite + Recharts + React Router 7
Backend	FastAPI + SQLAlchemy 2 + Pydantic Settings + PyJWT + pwdlib (Argon2)
Base de datos	PostgreSQL 17 (Supabase, alojada)
Despliegue	Render (API, serverless web service) · Vercel (web estática)
Pruebas	pytest (115) · Vitest + Testing Library (37) · ESLint

Cifras de código: **~9.300 líneas** en backend (78 módulos Python) y **~10.700** en frontend (TypeScript/TSX).

2. Arquitectura

```
  Navegador (React + TypeScript)
        │  HTTPS / REST + JWT
        ▼
  FastAPI  ── /api/auth        JWT, sesiones
           ── /api/customers, /products, /categories   catálogos
           ── /api/sales, /payments, /inventory        operación
           ── /api/datasets, /analyses                motor estadístico
           ── /api/analytics, /insights, /reports     inteligencia
           ── /api/audit                               trazabilidad
        │
        ├── app/services/   lógica de negocio (15 servicios)
        └── SQLAlchemy ──► PostgreSQL (23 tablas, aislamiento por empresa)
```

Principios aplicados:

- **Tres capas** (`api` → `services` → `models`): los routers validan y delegan; los servicios contienen la lógica; los modelos mapean el esquema.
- **Aislamiento multiempresa**: toda consulta pasa por `company_id` del usuario autenticado. Es la garantía central de seguridad entre clientes del SaaS.
- **Autorización en dos niveles**: el backend valida en cada petición (`app/core/permissions.py`); el frontend sólo oculta acciones que acabarían en 403 (`frontend/src/permissions.ts`). **El frontend nunca es autoridad.**
- **Password hashing Argon2** vía `pwdlib`; JWT con expiración de 60 min.

3. Estructura del repositorio

```
salesia-enterprise/
├── frontend/                React + Vite
│   └── src/
│       ├── App.tsx          rutas protegidas
│       ├── navigation.ts    menú lateral
│       ├── permissions.ts   control de acceso por rol (reflejo del backend)
│       ├── context/         AuthContext, SearchContext
│       ├── lib/api.ts       cliente HTTP con JWT y manejo de errores
│       ├── pages/           una página por módulo (+ tests)
│       ├── components/      MetricCard, Modal, Sidebar, Topbar
│       └── utils/format.ts  formato de moneda y fechas
├── backend/
│   ├── app/
│   │   ├── main.py          app FastAPI, CORS, routers, /health
│   │   ├── core/            config.py, security.py, permissions.py
│   │   ├── api/             routers (8 + 8 en routes/)
│   │   ├── services/        15 servicios de negocio
│   │   ├── models/          23 modelos ORM + base.py
│   │   └── schemas/         modelos Pydantic de entrada/salida
│   ├── tests/               6 archivos pytest (115 pruebas)
│   ├── seed_demo.py         seed reproducible para PostgreSQL
│   └── verify_faseNN.py     scripts de verificación end-to-end
├── database/
│   ├── migrations/001_initial_schema.sql   23 tablas (fuente de verdad)
│   └── seeds/                               datos de referencia
├── docs/                    documentación (este directorio)
└── render.yaml              blueprint de despliegue
```

> **La fuente de verdad del esquema es el SQL plano** en `database/migrations/`. Los modelos ORM lo reflejan; no se usa Alembic.

4. Configación y variables de entorno

`backend/app/core/config.py` define `Settings` (pydantic-settings), que lee `backend/.env` en local y las variables del entorno en producción.

Variable	Obligatoria	Descripción
`DATABASE_URL`	✔	Cadena SQLAlchemy. Producción: `postgresql+psycopg://…pooler.supabase.com:6543/postgres`
`JWT_SECRET_KEY`	✔	Firme del JWT. Generar con `openssl rand -hex 32`. **Sin default: el arranque falla si falta**
`JWT_ALGORITHM`		`HS256` (por defecto)
`JWT_ACCESS_TOKEN_EXPIRE_MINUTES`		`60`
`CORS_ORIGINS`	✔ prod	Orígenes permitidos, separados por coma. Debe incluir la URL del frontend
`APP_NAME` · `APP_ENV`		Metadatos; `APP_ENV=production` en despliegue

El frontend usa `VITE_API_BASE_URL` **en tiempo de build**. Vacío en desarrollo (usaría el proxy de Vite); en producción debe ser la URL absoluta de la API.

⚠️ `CORS_ORIGINS` debe casar **exactamente** el origen del navegador: sin barra final, sin comillas, sin espacios. Un desajuste produce el error típico *«Access-Control-Allow-Origin missing»*.

5. Puesta en marcha en local

```
# 1. Backend
cd backend
python3.12 -m venv .venv && .venv/bin/pip install -r requirements.txt
cp .env.example .env        # completar DATABASE_URL y JWT_SECRET_KEY
.venv/bin/python -m uvicorn app.main:app --reload --port 8000

# 2. Frontend (otra terminal)
cd frontend
npm install
npm run dev                 # http://localhost:5173, proxy a :8000
```

Credenciales de desarrollo: `admin@salesia.com` / `Admin123*`.

Para poblar una BD PostgreSQL vacía:

```
psql -f database/migrations/001_initial_schema.sql
psql -f database/seeds/001_reference_data.sql
psql -f database/seeds/002_admin_user.sql
cd backend && .venv/bin/python seed_demo.py   # o el seed completo de demo
```

6. Base de datos

23 tablas · 191 columnas. El detalle completo está en `diccionario-datos.md`. Consideraciones:

- **Claves primarias** `BIGINT` autogenerados (`GENERATED BY DEFAULT AS IDENTITY`). ⚠️ Un `INSERT` con `id` explícito **no avanza la secuencia**: tras un seed con IDs fijos hay que realinearla (`seed_demo.py::reset_sequences`, o `database/seeds/998_reset_sequences.sql`). Sin esto, el siguiente INSERT del API devuelve `409 duplicate key`.
- **`company_id`** en todas las tablas operativas: el filtro obligatorio.
- **Importes en `NUMERIC`** (decimal exacto); se envían/reciben como cadenas.
- **Fechas en `TIMESTAMPTZ`** (UTC).
- **`INET`** para IP de auditoría. ⚠️ PostgreSQL devuelve un objeto `ipaddress`, no `str`: hay que convertirlo antes de validar (`app/schemas/audit.py`). En SQLite no se manifiesta (el shim usa `String`).

7. Motor estadístico e insights

`app/services/stats_service.py` implementa media/mediana, variables aleatorias, probabilidad y **Bayes** (con posterior persistida en `bayes_analyses`). Se apoya en la librería estándar (`statistics`, `math`); los resultados se redondean a 4 decimales para que sean reproducibles.

`app/services/insight_service.py` define **12 reglas**, cada una con nombre, fuente y descripción. Las de origen comercial:

Código	Nombre
`revenue_trend`	Tendencia de ingresos
`category_concentration`	Concentración por categoría
`product_concentration`	Concentración por producto
`top_seller`	Vendedor destacado
`ticket_skew`	Sesgo del ticket promedio
`high_dispersion`	Alta dispersión de importes
`pending_sales`	Ventas pendientes
`cancellation_rate`	Tasa de cancelación

Las de origen estadístico: `mean_median_skew`, `random_variable_expected`, `probability_independence`, `bayes_posterior`.

Cada insight guarda una `evidence` **(JSONB)** con el dato numérico que lo origina y una `fingerprint` para no duplicar detecciones. Ese vínculo entre el insight y su evidencia es un requisito del plan.

8. Seguridad

- **Autenticación**: JWT (HS256) con `exp`; logout registra el evento.
- **Autorización**: matriz de 5 roles aplicada en `app/core/permissions.py` (ver `api.md`).
- **Auditoría**: `audit_logs` con acción, entidad, IP (`INET`) y detalle JSONB.
- **Validación**: Pydantic rechaza payloads inválidos con `422`.
- **Secretos**: `.env` en `.gitignore`; `JWT_SECRET_KEY` sin default.
- **CORS** restringido por variable de entorno.

9. Pruebas

```
cd backend  && .venv/bin/python -m pytest tests -q      # 115
cd frontend && npm run lint && npm test && npm run build # 37
```

La infraestructura de pruebas (`tests/conftest.py`) aplica *shims* de tipos para correr sobre un **SQLite temporal** y nunca toca la BD real. Ver `evidencias-pruebas.md`.

Scripts de verificación end-to-end (`verify_faseNN.py`) contra un backend en ejecución. **El orden recomendado es `12 → 11 → 10 → 9 → 8 → 13` y deben ejecutarse sobre una BD recién seedeada**: los scripts dejan datos residuales y algunos (Fase 11) calculan sobre deltas.

10. Despliegue

Blueprint en `render.yaml` (Render) + `backend/vercel.json` y `frontend/vercel.json` (Vercel). Ver `despliegue.md`.

11. Solución de problemas

Síntoma	Causa probable	Solución
`ValidationError: Field required` al arrancar	Falta `DATABASE_URL` o `JWT_SECRET_KEY`	Definir la variable en el entorno o en `.env`
`Access-Control-Allow-Origin` faltante	`CORS_ORIGINS` no casa el origen exacto	Sin barra final/comillas; redeploy tras guardar
`500` en `/api/audit` sobre Postgres	`INET` devuelto como `IPv4Address`	Mantener el `field_validator` de `app/schemas/audit.py`
`409 duplicate key` tras un seed	Secuencia `IDENTITY` desalineada	Ejecutar `reset_sequences`

Síntoma	Causa probable	Solución
Peticiones que fallan con el pooler	Conexión ociosa cerrada por Supabase	`pool_pre_ping=True` ya está activo; revisar puerto 6543
Frontend en blanco / 404 en recarga	Falta el rewrite SPA	`frontend/vercel.json` debe estar presente
Pruebas que tocan datos reales	`DATABASE_URL` no sobreescrita	`conftest.py` debe fijar el SQLite temporal antes de importar la app